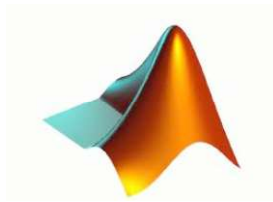
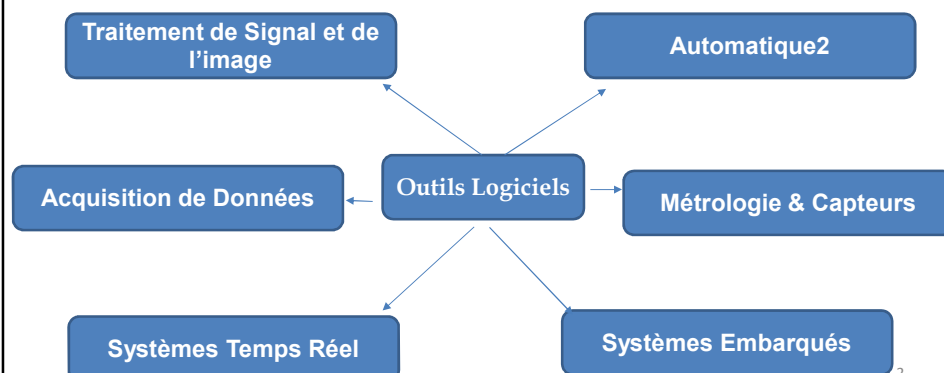

Formation matlab



Objectif

- Fournir les outils informatiques nécessaires à la conception et la simulation de systèmes complexes en électronique.
- Se familiariser avec les environnements de chaque logiciel.
- Comprendre et utiliser la notion de bibliothèque.
- Créer les propres modèles, les simuler et analyser leurs résultats.



Partie I- Matlab

plan

- I. Introduction
- II. Environnement : syntaxe
- III. Objets de base : variables et leurs types
- IV. Déclaration de signaux
- V. Opérations sur les signaux

3

I- Introduction

Pourquoi programmer ?

- **Automatisation de tâches répétitives**
 - analyse de signaux reçus par un sonar de sous-marin
 - calculs de trajectoire
 - test de résistance d'un matériau soumis à des vibrations aléatoires,
 - affichage de plusieurs résultats dans le même formatParce qu'un ingénieur doit donner une **réponse vérifiable** dans un **délai court** à un problème posé, incorporant souvent de **nombreuses variables/valeurs**
- Parce que les **données** d'un problème peuvent **varier** dans le temps
- Parce qu'il n'existe pas de programme tout fait qui permet de faire exactement ce que l'on veut

4

Les étapes de la programmation pour résoudre un problème non trivial

0. Identification et spécification du problème

1. Modélisation du problème (organigramme)

1' Subdivision du pbm en **tâches simples** et indépendantes

2. Planification des **algorithmes** en **pseudo-code** (le plan d'attaque pour résoudre le problème)

3. Traduction en langage de programmation (Matlab)

4. Commenter ce code au fur et à mesure

5. Tester le programme **par petits bouts** et traquer les erreurs

6. Utiliser !

5

Exercice

Calculer le volume suivant:

$$V = \frac{4}{3}\pi R^2 \text{ ou } R=4\text{cm}$$

Dans le domaine de l'analyse numérique de données, on a souvent besoin d'établir un modèle mathématique liant plusieurs série de données expérimentales.

6

Question.. Matlab ?

Qu'est-ce que Matlab?

Langage matriciel de haut niveau, performant, puissant pour l'informatique technique, efficace pour tout traitement mathématique de données
 Il possède des structures de contrôle, des fonctions, des structures de données, des fonctionnalités d'E / S.
 Permet de créer des programmes d'application volumineux et complexes

Quelles sont les fonctions de gestion de la mémoire dans Matlab?

- **Clear** - Removes variables from memory.
- **Pack** - Enregistre les variables existantes sur le disque, puis les recharge de manière régulière.
- **Save** - Selectively persists variables to disk.
- **Load** - Reloads a data file saved with the save function.
- **Quit** - Exits MATLAB and returns all allocated memory to the system.

Savez-vous ce qu'est Simulink?

Un produit complémentaire à MatLab, il fournit un environnement graphique interactif de simulation pour la modélisation et l'analyse de systèmes dynamiques.

7

Pourquoi utiliser Matlab ?

- C'est un environnement de programmation **facile à utiliser**
 - Plusieurs **fonctions prédéfinies** pour analyser et représenter des données : on peut faire des choses élaborées avec très peu de code.
 - Particulièrement **adapté à la résolution de problèmes numériques** rencontrés par les ingénieurs
 - Il contient des outils d'optimisation et de résolution d'équations
 - Il contient des modules spécialisés d'analyse du signal et de l'image
 - Disponible dans les salles de TP et en entreprise
- !!**  La partie graphique est pauvre --- LabView

II- Environnement : syntaxe

Qu'est-ce qu'un environnement ?

Un environnement est un simple dossier de votre disque dur. Il faut donc indiquer à MatLab dans quel dossier se placer. C'est dans ce dossier que MatLab ira chercher les fonctions que vous avez définies, ainsi que le script.

Comment se placer dans le bon environnement ?

- Les principales fenêtres
- L'aide

9

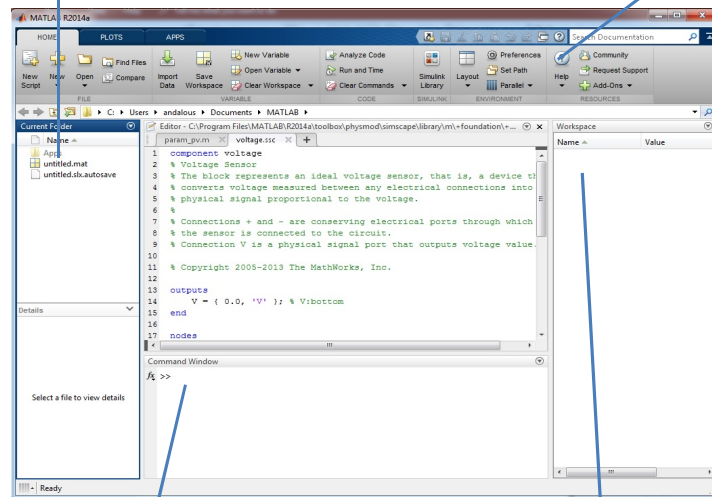
C'est quoi l'utilité du Matlab?

1. Calculs simples
2. Tracer et analyser des relations mathématiques (2D et 3D)
3. Opérations sur listes et matrices
4. Ecrire des fichiers de script (un type de programmation)
5. Manipulation symbolique des équations
6. Visualisation avancée, animation et outils d'interface graphique

10

Gestionnaire de fichier :
liste les fichiers du répertoire de travail

Obtenir de l'aide



Invite de commandes :
Entrer les commandes à exécuter

Workspace :
liste les variables en mémoire 11

III- Objets de base : variables et leurs types

Un algorithme est une suite d'instructions manipulant des **informations** :



Variables

- Une variable est un **emplacement mémoire** pour stocker provisoirement une donnée, repéré par un **nom**. Cet emplacement est composé d'**une ou plusieurs cases**, chaque case ne pouvant contenir qu'**une seule valeur**

X**T**

13

Types

- Chaque variable a un **type** qui indique à l'ordinateur comment décoder l'information située dans l'emplacement et comment la manipuler dans des opérations.

- nombres entiers, réels, booléens, chaînes,...

Chaque type a des **opérations spécifiques** :

- +, -, /, * pour les nombres
- sous-chaîne pour les chaînes de caractères

14

Types en Matlab

- Opérateurs et fonctions logiques :

Function	Operation
<code>true, false</code>	Setting value to true or false
<code>logical</code>	Numeric to logical conversion
<code>& (and), (or), ~ (not), xor, any, all</code>	Logical operations
<code>&&, </code>	Short-circuit AND and OR
<code>== (eq), ~= (ne), < (lt), > (gt), <= (le), >= (ge)</code>	Relational operations
All <code>is*</code> functions, <code>cellfun</code>	Test operations
<code>strcmp, strncmp, strcmpi, strncmpi</code>	String comparisons

15

Les fichiers et la programmation avec MATLAB

16

Dans MATLAB, il y a deux types de fichier : les fichiers de données et les fichiers de programmes (scripts et fonctions).

Fichiers de données

En plus des fichiers de données que l'on peut définir et utiliser par programmation, dans MATLAB on trouve les fichiers MAT. Ce sont des fichiers binaires (d'extension « mat ») qui permettent de stocker et de restituer des variables utilisées dans l'espace de travail. Ces opérations sont réalisées respectivement par les commandes *save* et *load*.

Exemple (sauvegarde de variables)

On définit une variable *t*.

```
>> t = [1 2 3]
t = 1    2    3
```

On sauvegarde la variable *t* dans le fichier *fic_t*,

```
>> save fic_t t
```

17

Le fichier obtenu aura pour extension « **mat** » et sera sauvegarder sous le nom ***fic_t.mat***.

```
>> clear all
```

MATLAB ne connaît plus la variable *t*.

```
>> t
??? Undefined function or variable 't'.
```

Si l'on charge le fichier *fic_t*, la variable *t* est de nouveau présente dans l'espace de travail.

```
>> load fic_t
```

Cette vérification peut se faire, soit par appel de cette variable,

```
>> t
t = 1    2    3
```

Soit par l'utilisation de la commande *whos*

```
>> whos
Name      Size      Bytes Class  Attributes
t         1x3         24 double
```

18

Lorsque les problèmes deviennent compliqués et nécessitent une re-évaluation, entrer la commande à l'invite MATLAB n'est pas pratique

Solution : use **M-files**

Script

- Collections de commandes
- Exécuté en séquence quand il est appelé
- Enregistré avec extension ".m"

Function

- Commandes définies par l'utilisateur
- Définie l'entrée et la sortie
- Enregistré avec extension ".m"

19

La fonction plot.

plot permet de tracer une ou plusieurs courbes sur le même graphe, en choisissant le style du trait et sa couleur et en marquant les points du tableau des valeurs si on le désire.

```
ex :      x = linspace(0,2*pi,30);
          y = sin(x); z = cos(x);
          plot(x,y,'g:',x,z,'r--',x,y,'mo',x,z,'c+');
```

Les commandes grid, xlabel, ylabel et text.

grid on met la grille sur le graphe tracé par plot. **grid off** efface la grille.
xlabel et **ylabel** écrivent du texte le long de l'axe correspondant.

Les commandes figure, hold, close....

20

Valeurs et opérateurs logiques

TP

Manipuler des variables

$A \leftarrow 0$ on affecte la valeur 0 à la variable A, si A n'existait pas auparavant, elle est créée.
 $B \leftarrow 11$ on affecte la valeur 11 à la variable B

On peut alors utiliser une variable dans un calcul ou pour affecter son contenu à une autre variable.

$A \leftarrow B$ on affecte la valeur contenue ds B à la variable A, A vaut à présent 11, B reste inchangée
 $A \leftarrow A + 1$ on affecte la valeur contenue ds A incrémentée de 1 à la variable A, A vaut à présent 12

Sous MATLAB, l'affectation se fait à l'aide de l'opérateur =

```
>> A = 0
>> B = 11
>> A = B
>> A = A + 1
```

21

Caractères réservés : % = ; ,

TP

- Certains caractères sont réservés par MATLAB à des fins diverses. Certains comme opérateurs arithmétiques ou matriciels : =, +, -, *, /, \ et d'autres sont utilisés pour effectuer une multitude d'opérations. Les caractères réservés ne peuvent pas être utilisés dans les noms de variables ou de fonctions.
- >> % anything after % until the end of line is treated as comments
- >> a = 3 % define a to have the value 3
- a = 3
- >> a = 3; % ";" suppresses printing
- >> b = 4; c = 5; % ";" enables multiple commands on same line
- >> d = 6, e = 7; % "," delimits commands but enables printing
- d = 6

22

TP

Caractères réservés : [] ()

- `>> x = 1:2:9` % define vector x with : operator (begin:interval:end)
`x = 1 3 5 7 9`
- `>> y = 3:5` % interval is defaulted to 1; same as `y=[3:5]`
`y = 3 4 5`
- `>> X = [1, 2, 3; 4, 5, 6]` % 2D array. The ; is vertical concatenation.
 % [] for arrays. Prevents ambiguity
 % ; concatenates vertically (new row)
 % , concatenates horizontally (new columns)
`X =`
`1 2 3`
`4 5 6`
- `>> X(2,3)` % () for subscripting; why ans ?
`ans = 6`

23

TP

Caractères réservés: . . . et '

- `>> x = [1 2 3 ...` % ellipses ... means to be continued on the next line
`4 5 6]`
`x = 1 2 3 4 5 6`
- `>> s = 'this is a character string';` % blanks preserved within quotes
- `>> x = [1 2 3]'` % ' performs transpose (e.g., turns row into column)
`x =`
`1`
`2`
`3`
- `>> X = [1 2 3; 4 5 6]; size(X)` % figure out the size (**dimensions**) of X
`ans = 2 3`
- `>> X = [1 2 3; 4 5 6]; numel(X)` % **total** number of entries in X
`ans = 6`

24

Opérations de tableau

TP

```
>> a = 1:3;      % a is a row vector
>> b = 4:6;      % b is a row vector
>> c = a + b      % c has same shape as a & b
c = 5    7    9

>> A = [a;b]      % combines rows to generate 2x3 matrix A;
A=a;b ?
A =
     1     2     3
     4     5     6
>> B = A'          % B is transpose of A
B =
     1     4
     2     5
     3     6
Other ways to create B ? (hint: with a and b )
```

25

Opérations matricielles

TP

```
>> C = A*B        % * is overloaded as matrix multiply operator
C =
    14    32
    32    77
>> D = A.*A        % a .* turns matrix multiply to elemental
multiply
D =
     1     4     9
    16    25    36
>> E = A./A        % elemental divide
E =
     1     1     1
     1     1     1
>> who            % list existing variables in workspace
Your variables are:
A B C D E a b d
```

26

Précision des données

TP

>> whos % detail listing of workspace variables

Name	Size	Bytes	Class	Attributes
A	2x3	48	double	
B	3x2	48	double	
C	2x2	32	double	
D	2x3	48	double	
E	2x3	48	double	
a	1x3	24	double	
b	1x3	24	double	
c	1x3	24	double	

>> A = single(A); % recast A to single data type to save memory

>> whos

Name	Size	Bytes	Class
A	2x3	24	single

>> clear % delete all workspace variables

27

• Rappel

Calculer le volume suivant:

$$V = \frac{4}{3}\pi R^2 \text{ ou } R=4\text{cm}$$

28

• Rappel

- Les commandes Matlab doivent toujours être tapées en **minuscules** même si dans l'aide en ligne elles apparaissent en majuscules.
- **u1=1:10** (*incrémentation automatique de 1 à 10 avec pas de 1*)
- Input(' '): introduire une ou des variables par intermédiaires du clavier
`a=input('donner la valeur de a : ');`
- affichage Function:
`disp(var)`
`disp('Output string')`
- Number to String Function:
`num2str(var)`

29

• Rappel

- **Un=ones(2)** (*matrice constituée de 1 d'ordre 2*)
`Un =`
`1 1`
`1 1`
- **Z=zeros(2,3)** (*matrice nulle d'ordre 2x3*)
`Z =`
`0 0 0`
`0 0 0`

30

• Rappel

TP

Définir une matrice.

On utilise les crochets [et] pour définir le début et la fin de la matrice. On peut également utiliser le symbole , qui sert de séparateur de colonne et ; de séparateur de ligne.

A = [1 2 3 4] définit la **matrice ligne** comportant les 4 éléments 1, 2, 3 et 4.
 A = [1,2,3,4] donne le même résultat.
 C = ['a' 'b' 'c'] définit la **matrice ligne** comportant les caractères a, b et c.
 B = [1;2;3;4] définit la **matrice colonne** comportant les mêmes valeurs que A. Si on remarque que B est la transposée de A, on peut aussi écrire B = A' l'apostrophe à droite signifiant la **transposition**.
 C = [1 2 3;4 5 6;7 8 9] définit la **matrice carrée** 3x3, dont la première ligne est 1 2 3, la seconde ligne 4 5 6, etc... On obtient sa **transposée** en écrivant **C'**.

31

• Rappel

➤ Il existe des syntaxes particulières permettant d'extraire des lignes ou des colonnes de matrices :

➤ **A1=[11 12 13;21 22 23;31 32 33]**

A1 =
 11 12 13
 21 22 23
 31 32 33

➤ **A1(:,1)** (colonne 1 de la matrice A1)

ans =
 11
 21
 31

➤ **A1(2,:)** (ligne 2 de la matrice A1)

ans =
 21 22 23

➤ **clear all** : efface tous les objets en mémoire

32

• Rappel

TP

Problème avec deux Variables

- | | |
|---|--|
| <ul style="list-style-type: none"> • Two Scalars <ul style="list-style-type: none"> – <code>x = 3;</code> – <code>y = 5;</code> – <code>A = x * y</code> – <code>A = 15</code> • Scalar and Vector <ul style="list-style-type: none"> – <code>x = 1:5;</code> – <code>y = 5;</code> – <code>A = x * y</code> – <code>A = 5 10 15 20 25</code> | <ul style="list-style-type: none"> • Two Vectors <ul style="list-style-type: none"> – <code>x = 1:5;</code> – <code>y = 1:0.5:3;</code> – <code>A = x * y</code> – <code>Error</code> • Two Vectors (by element) <ul style="list-style-type: none"> – <code>x = 1:5;</code> – <code>y = 1:0.5:3;</code> – <code>A = x .* y</code> – <code>A = 1 3 6 10 15</code> |
|---|--|

33

• Rappel

Exercices

- Utiliser des virgules au lieu d'espaces entre les éléments vectoriels si vous le souhaitez. Essayez ça :

$$a = [5,6,7]$$
- N'oubliez pas les virgules (ou espaces) entre les éléments; sinon, vous pourriez vous retrouver avec quelque chose de très différent:

$$x = [130-15]$$

Que pensez-vous que cela donne?

- Vous pouvez utiliser un vecteur dans une liste pour un autre. Tapez ce qui suit:

$$a = [1\ 2\ 3];$$

$$b = [4\ 5];$$

$$c = [a\ -b];$$

Pouvez-vous déterminer à quoi ressemblera c avant de l'afficher??

- Que pensez-vous que cela donne?

$$a = [1\ 3\ 7];$$

$$a = [a\ 0\ -1];$$

Rappelez-vous les règles importantes suivantes:

- ❖ Les éléments de la liste doivent être placés entre crochets et non entre parenthèses.
- ❖ Les éléments de la liste doivent être séparés par des espaces ou par des virgules.

• Rappel

Exercices

1. Enter `r = rand(1,7)`. Cela vous donne un vecteur de ligne de sept nombres aléatoires.
2. Enter `r(3)`. Cela affichera le troisième élément de `r`. Le chiffre 3 est l'indice.
3. Enter `r(2:4)`. Cela devrait vous donner les deuxième, troisième et quatrième éléments.
4. What about `r(1:2:7)` and `r([1 7 2 6])`?
5. Utilisez un vecteur vide pour supprimer des éléments d'un vecteur:
`r([1 7 2]) = []`
Cela va supprimer des éléments 1, 7, et 2.

Résumer:

- ❖ Un indice est indiqué entre parenthèses.
- ❖ Un indice peut être un scalaire ou un vecteur.
- ❖ Dans MATLAB, les indices commencent toujours à 1.

35

• Rappel

Opérations arithmétiques entre deux scalaires

Operation	Algebraic form	MATLAB
Addition	$a + b$	<code>a + b</code>
Soustraction	$a - b$	<code>a - b</code>
Multiplication	$a \times b$	<code>a * b</code>
Division	a/b	<code>a / b</code>
Puissance	a^b	<code>a ^ b</code>

La priorité des opérations dans une expression

Les opérations	La priorité (1=max, 4=min)
Les parenthèses (et)	1
La puissance et le transposé ^ et '	2
La multiplication et la division * et /	3
L'addition et la soustraction + et -	4

36

• **Rappel**

Opérateurs de comparaisons

Opération	Forme algébrique	MATLAB
égal à	= =	x == y
strictement plus grand que	>	x > y
strictement plus petit que	<	x < y
plus grand ou égal à	>=	x >= y
plus petit ou égal à	<=	x <= y
différent de	~=	x ~= y

Opérateurs arithmétiques pour les tableaux élément par élément

Opérateurs	Description
.*	Multiplication
.^	Power
./	Division

37

• **Rappel**

Utilisez MATLAB pour évaluer les expressions suivantes

(a) $\frac{1}{2*3}$ (a) (0.1667)

(b) 2^{2*3} (b) (64)

(c) $1.5*10^{-4} + 2.5*10^{-2}$ (c) (0.0252)

Proposer des instructions Matlab pour engendrer les vecteur suivants:

V1=[2,3,4,.....,9,10]

Création des vecteurs

>> V1=[2:10]

V2=[-1.5,0,1.5,....,4.5,6]

>> V2=[-1.5:1.5:6]

38

Script m-file

TP

Si vous avez un groupe de commandes devant être exécuté de manière répétée, il convient de les enregistrer dans un fichier. . .

```
>> edit mytrig.m % enter commands in editor window
a=sin(x); % compute sine x (radians)
b=cos(x); % compute cosine x (radians)
disp(['a = ' num2str(a) ]) % prints a; here, [ . . . ] constitutes a string
array
disp(['b = ' num2str(b) ]) % prints b
```

Select File/Save to save it as mytrig.m.

Un script partage la même étendue que celui avec lequel il opère. Par exemple, s'il fonctionne à partir du matlab

Define x, then use it in mytrig.m:

```
>> x=30*pi/180; % converts 30 degrees to radians
>> mytrig % x is accessible to mytrig.m; share same workspace
a = 0.5000
b = 0.8660
```

Function m-files

TP

- Il est déclaré avec la fonction mot clé, avec des paramètres d'entrée optionnels sur la sortie droite et facultative sur la gauche de =. Tous les autres paramètres de la fonction résident dans l'espace de travail de la fonction.

Use MATLAB editor to create file: >> edit average.m

```
function avg=average(x)
% function avg=average(x)
% Computes the average of x
% x (input) matrix for which an average is sought
% avg (output) the average of x
avg = sum(x)/numel(x); % the average
end
```

Save the above with File/Save

- Recommendation: enregistre le fichier avec le même nom que la fonction
- Il peut être appelé depuis un script, une autre fonction ou sur la ligne de commande :
- >> a = average(1:3) % a = (1 + 2 + 3) / 3


```
a =
     2
```
- >> help average % prints contiguous lines with % in average

Script or Function m-file ?

TP

Scripts

- **Avantages:**
 - pratique; Les variables du script sont dans le même espace de travail que celui de l'appelant.
- **Inconvénients :**
 - lent; commandes de script chargées et interprétées chaque fois qu'il est utilisé
 - les risques de conflit de noms de variables à l'intérieur et à l'extérieur du script

Functions

- **Avantages:**
 - La portée des variables de la fonction se limite à la fonction. Aucun souci pour le nom n'entre en conflit avec ceux en dehors de la fonction.
 - Ce qui entre et sort est étroitement contrôlé, ce qui aide lorsque le débogage devient nécessaire.
 - Compilé la première fois qu'il est utilisé; s'exécute plus rapidement les fois suivantes.
 - Facilement être déployé dans un autre projet.
 - Nettoyage automatique des variables temporaires.
- **Inconvénients:**
 - Les E / S sont hautement réglementées, si la fonction nécessite de nombreuses variables prédéfinies, il est fastidieux de faire entrer et sortir de la fonction - un fichier m-script est plus pratique.

Some Frequently Used Functions

TP

```
>> magic(n)    % creates a special  $n \times n$  matrix; handy for testing
>> zeros(n,m)  % creates  $n \times m$  matrix of zeroes (0)
>> ones(n,m)   % creates  $n \times m$  matrix of ones (1)
>> rand(n,m)   % creates  $n \times m$  matrix of random numbers
>> repmat(a,n,m) % replicates  $a$  by  $n$  rows and  $m$  columns
>> diag(M)     % extracts the diagonals of a matrix M
>> help elmat  % list all elementary matrix operations ( or elfun)
>> abs(x);     % absolute value of  $x$ 
>> exp(x);     %  $e$  to the  $x$ -th power
>> fix(x);     % rounds  $x$  to integer towards 0
>> log10(x);   % common logarithm of  $x$  to the base 10
>> rem(x,y);   % remainder of  $x/y$ 
>> mod(x, y);  % modulus after division - unsigned rem
>> sqrt(x);    % square root of  $x$ 
>> sin(x);     % sine of  $x$ ;  $x$  in radians
>> acoth(x)    % inversion hyperbolic cotangent of  $x$ 
```

Scripts et fonctions

Le script est le programme principal. Parfois, c'est même le seul élément du programme.

Le principe est simple : regrouper dans un fichier une série de commandes MATLAB et les exécuter en bloc. Tout se passera comme si vous les tapiez au fur et à mesure dans une session MATLAB. Il est fortement conseillé de procéder comme ceci. Cela permet notamment de récupérer facilement votre travail de la veille.

Les fichiers de commandes peuvent porter un nom quelconque, mais doivent finir par l'extension **".m"**.

Comment créer un script ?

Pour créer un fichier de commandes, taper des commandes MATLAB à l'intérieur de l'éditeur intégré, sauver par exemple sous le nom "toto.m" .

Pour exécuter un script, dans la fenêtre de commande MATLAB, il suffit de mettre son nom après le prompt (>>) (exemple >> **toto**) ou de cliquer sur la flèche verte de l'éditeur . L'éditeur de MATLAB permet de déboguer le programme, facilite l'édition en affichant des couleurs différentes selon le type de données (commentaires en vert, chaîne de caractères en violet, etc.).

43

Scripts et fonctions

Les fonctions ne sont pas indispensables au bon fonctionnement d'un programme. Mais elles améliorent la lisibilité, et évitent de taper de nombreuses fois les mêmes instructions. D'autre part, MatLab n'a pas d'instructions du type GoTo (c'est une commande en basic qui permet d'aller d'un point à un autre du programme), les fonctions peuvent donc se substituer à ce type d'instruction.

La fonction doit commencer par la syntaxe suivante :

function [variables de sortie] = nomdelafunction (variables d'entrée)

44

Scripts et fonctions

La fonction doit finir par la commande : **end**

Exemple

```
function [A,B]=produitetsomme(alpha,beta,gamma)
    opérations sur alpha, beta, gamma affectation des
    résultats des opérations à A et B
end
```

45

Boucle *for*



Parcours d'un intervalle:

une première possibilité pour exécuter une séquence d'instructions de manière répétée consiste à effectuer une boucle pour les valeurs d'indice, incrémenté à chaque iteration, variant entre deux bornes données. Ce processus est mis en oeuvre par la boucle *for*.

syntaxe:

```
for indice= borne_inf : borne_sup
    séquence d'instructions
end
```

46

Boucle *for*

TP

Si la boucle *for* a la forme

for k = first : increment : last

Le nombre de fois que la boucle est exécutée peut être calculé à

partir de l'équation suivante: $\left(\frac{\text{last} - \text{first}}{\text{increment}}\right) + 1$

Cette valeur s'appelle l'itération ou le nombre de déclenchements. À titre d'exemple, considérons l'énoncé:

for i = 1: 2: 6. Il a un nombre d'itérations de

$$\left(\frac{6-1}{2}\right) + 1 = 3$$

Ainsi, *i* prends les valeurs 1, 3, 5. Notez que si le nombre d'itérations est négatif, la boucle n'est pas exécutée.

47

Boucle *for*

TP

Exemple

- *for* i = 1:5, disp(i), end
- *for* i = 1:3, disp(i), end
- *for* i = 1:0, disp(i), end

48

Boucle *for*

TP

Exemple trois formes de l'instruction

L'instruction <i>for</i>	<i>for</i> i=1:4 j=i*2; disp(j) end	<i>for</i> i=1:2:4 j=i*2; disp(j) end	<i>for</i> i=[1,4,7] j=i*2; disp(j) end
résultat	2 4 6 8	2 6	2 8 14

49

Boucle *for*

TP

Exemple: Racines carrées avec la méthode de Newton

La racine carrée x de tout nombre positif a peut être trouvée en utilisant uniquement les opérations arithmétiques d'addition, de soustraction et de division avec Méthode de Newton. C'est une procédure itérative (répétitive) qui affine une initiale deviner.

1. Initialiser a
2. Initialiser x à $a/2$
3. Repetter 6 fois (say)
 - Remplacer x par $(x + a/x)/2$
 - afficher x
4. Stop

50

Boucle for

TP

Exemple: Racines carrées avec la méthode de Newton

```
a=input('donner la valeur de a : ');
%a = 2;
x = a/2;
disp(['The approach to sqrt(a) for a = '])

    for i = 1:6
        x = (x + a / x) / 2;
    disp( x )
    end
disp( 'Matlab''s value: ' )
disp( sqrt(2) )
```

51

Boucle for

TP

Exemple: Factorials!

Proposer le programme pour générer une liste de n et n!

$$n! = 1 \times 2 \times 3 \times \dots \times (n - 1) \times n$$

```
n=input('donner la valeur de n : ');
%n = 10;
fact = 1;
```

```
    for k = 1:n
        fact = k * fact ;
        disp( [k fact] )
    end
```

52

if Conditionnée

TP

On a parfois besoin de d'executer une sequence d'instructions seulement dans le cas où une condition donnée est vérifiée au préalable. Différentes forms d'instruction conditionnée existent sous matlab.

syntaxe:

if expression logique
séquence d'instructions
end

Où:

- ✓ *expression logique* est une expression dont le resultat peut être vrai ou faux;
- ✓ *séquence d'instructions* est le traitement à effectuer si *expression logique* est vrai. mais si la condition est fausse, rien ne se produit.

53

if Conditionnée

TP

On a parfois besoin d'executer une sequence d'instructions seulement dans le cas où une condition donnée est vérifiée au préalable.

syntaxe:

if expression logique
séquence d'instructions 1
else
séquence d'instructions 2
end

Où:

- ✓ *expression logique* est une expression dont le resultat peut être vrai ou faux;
- ✓ *séquence d'instructions 1* est le traitement à effectuer si *expression logique* est vrai. mais si la condition est fausse, c'est *séquence d'instructions 2* qui est exécutée,

54

if Conditionnée

TP

Exemple: un programme qui vous définit selon votre age

```
age=input('Entrez votre âge: ');...
if (age <2)
    disp('votre êtes un bebe')
elseif (age <13)
    disp('votre êtes un enfant')
elseif (age <18)
    disp('votre êtes un adolescent')
elseif (age <60)
    disp('votre êtes un adult')
else
    Disp('vous êtes un vieillard')
end
```

if Conditionnée

TP

Exemple: un programme qui trouve les racine d'une équation 2 degré

```
a=input ('Entrez la valeur de a : '); %lire a
b=input ('Entrez la valeur de b : '); %lire b
c=input ('Entrez la valeur de c : '); %lire c

delta=b^2-4*a*c; %calculer delta
if delta<0
    disp('Pas de solution') %pas de solution
elseif delta==0
    disp('Solution double: ') %solution double
    x=-b/(2*a)
else
    disp('Deux solutions distinctes: ') %deux solution
    x1=(-b+sqrt(delta))/(2*a)
    x1=(-b-sqrt(delta))/(2*a)
end
```

Boucle *while*: tant que...faire

TP

Une autre possibilité pour exécuter une séquence d'instructions de manière répétée consiste à effectuer une boucle tant qu'une condition reste vérifiée. On arrête de boucler dès que cette condition n'est plus satisfaite. Ce processus est mis en oeuvre par la boucle *while*.

Si on ne connaît pas le nombre d'itérations nécessaire à l'avance, une boucle *while* est préférable par rapport à une boucle *for*.

syntaxe:

```
while expression logique
    séquence d'instructions
end
```

57

Boucle *while*:

TP

Exemple

```
n=input('donner la valeur de n : ');
k=1; nfac=1;
while k<=n
    nfac=nfac*k;
    k=k+1;
    disp(nfac);
end
disp( ['le fact de n est ' num2str(nfac)] );
```

58

Instruction switch

TP

Une alternative à utilisation d'une séquence d'instructions conditionnées pour effectuer un choix en seconde existe. Il s'agit de l'instruction **switch**.

syntaxe:

```
switch var
    case cst1,
        séquence d'instructions 1
    case cst2,
        séquence d'instructions 2
    .....
    case cstN,
        séquence d'instructions N
otherwise
    séquence d'instructions par défaut
end
```

Où:

- ✓ **var** est une variable numérique ou une variable chaîne de caractères;
- ✓ **cst1**, ..., **cstN**, sont des const numérique ou une constantes chaîne de caractères;
- ✓ *séquence d'instructions i* est la séquence d'instructions à exécuter si le contenu de la variable var est égal à la cst csti (var==csti)

- Il est possible de regrouper plusieurs "cas" si la séquence d'instructions à exécuter est la même pour ces différents cas. La syntaxe est alors:

```
case {cst1, cst1,...}
    séquence d'instructions commune
```

59

Instruction switch

TP

Exemple

```
k=input('donner k ');
switch(k)

    case 1
        disp('case 1 ');
    case 3
        disp('case 3 ');
    case 5
        disp('case 5 ');
    otherwise
        disp('case n'est pas 1 ou 3 ou 5 ');
end
```

60

Instruction break

TP

L'instruction *break* provoque l'arrêt de la première boucle *for*, *while*

L'instruction *for* ci-dessous est stoppée au premier *i* tel que *t(i)* est nul:

```
t=input('donner une valeur T:');
t=-2:10;
for i=1:length(t)
    if t(i)==0
        break;

    end
    % disp(['i= ',num2str(i)])
    % disp(['t(i)= ',num2str(t(i))])
end
%disp(['i= ',num2str(i)])
%disp(['t(i)= ',num2str(t(i))])
```

61

Instruction continue

TP

L'instruction continue: dans une instruction *for*, *while*, l'instruction continue provoque l'arrêt de l'itération courante, et le passage au début de l'itération suivante.

Supposons que l'on parcoure un tableau *t* pour réaliser un certain traitement sur tous les éléments, sauf ceux qui sont négatifs

```
t=input('donner une valeur T:');
t=-2:10;
for i=1:length(t)
    if (t(i)<0)
        continue;           % on passe au i suivant dans le for

    end
    ...                     % traitement de l'élément courant
end
```

62

Exercice récapitulatif

Des fonctions prédéfinis en MATLAB, essayer de les programmer (un vecteur donnée V)

La fonction `sum(V)` : la somme des éléments d'un vecteur V

```
V=input('donner une vecteur V : ');
n=length(V);
somme=0;
for i=1:n
    somme=somme+V(i);
end
disp(somme)
```

La fonction `prod(V)` : le produit des éléments d'un vecteur V

```
V=input('donner une vecteur V : ');
n=length(V);
produit=1;
for i=1:n
    produit=produit*V(i);
    disp(produit)
end
```

63

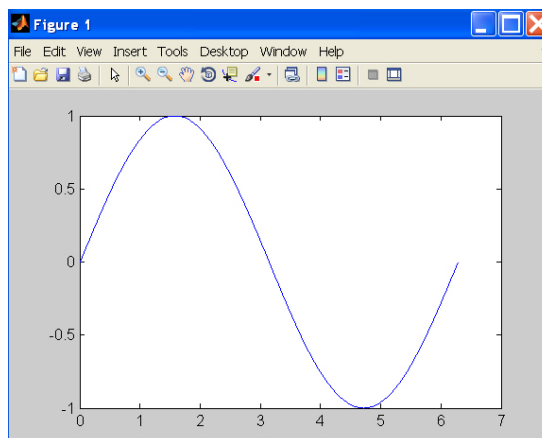
Line Plot

TP

L'utilisation la plus simple de l'instruction `plot` est la suivante.

On souhaite afficher la fonction $y = \sin(t)$ sur $0 < t < 2\pi$

```
>> t = 0:pi/100:2*pi;
>> y = sin(t);
>> plot(t,y)
```



Line Plot

TP

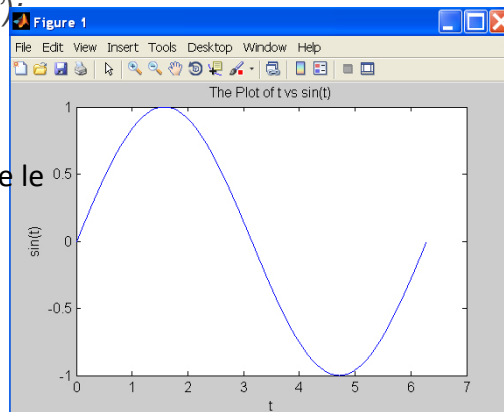
Pour rajouter un titre et des labels d'axes au figures, on utilise

```
>> title('The plot of t vs sin(t)');
>> xlabel('t');
>> ylabel('sin(t)');
```

xlabel et **ylabel** écrivent du texte le long de l'axe correspondant.

Une grille en pointillés peut également être rajouter

```
>> grid on;    % affichage de la grille
>> grid off    % masquage de la grille
```



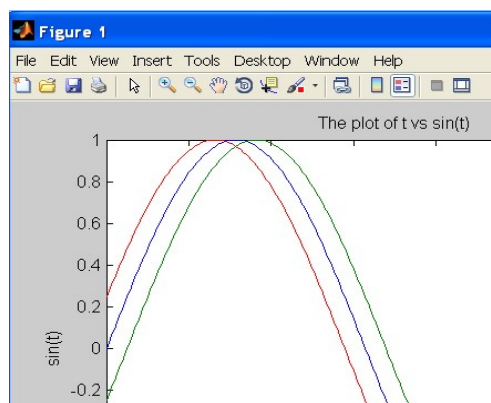
Line Plot

TP

Style de ligne et couleurs

```
>> plot(t,y,'g.')
```

```
>> y2 = sin(t-0.25);
>> y3 = sin(t+0.25);
>> plot(t,y,t,y2,t,y3) % make 2D line plot of 3 curves
>> legend('sin(t)', 'sin(t-0.25)', 'sin(t+0.25)', 1)
```



EXERCICES D'APPLICATION.

TP

EXERCICE :

Proposez un exemple de script, stocké dans un fichier appelé *courbel.m*, dont le code permet de tracer la courbe de la fonction $y = x^2 + 5$, sur l'intervalle $[-5, 5]$.

La courbe doit être affichée avec :
grid , title, xlabel , ylabel

67

EXERCICES D'APPLICATION.

TP

Exercice 1:

- 1-Écrire la fonction (function) dans le fichier fct1.m.
- 2- Tracer la courbe correspondant (avec grid , title, xlabel , ylabel) à cette fonction entre -50 et 50.

Exercice 2:

- 1-Écrire un script dans le fichier fct2.m.

$$y = \sin(x^2) + 2x + \frac{2}{x+1}$$

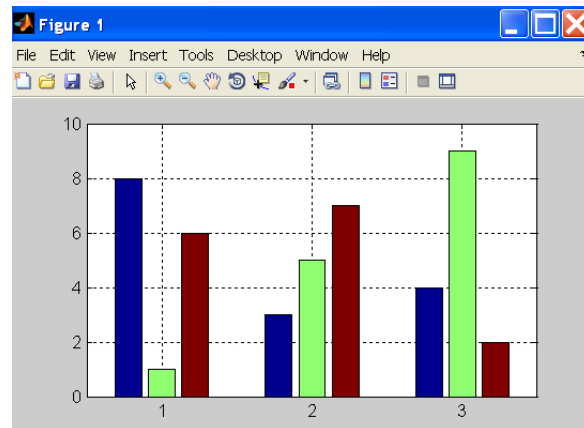
- 2- Tracer la courbe correspondant (avec grid , title, xlabel , ylabel) à cette fonction entre -5 et 5.

68

2D Bar Graph

TP

```
>> x = magic(3); % generate data for bar graph
>> bar(x)        % create bar chart
>> grid          % add grid for clarity
```



Use MATLAB Command or Function ?

TP

- De nombreux utilitaires MATLAB sont disponibles dans les formulaires de commande et de fonction.
- Pour cet exemple, les deux formes produisent le même effet:
- `>> print -djpeg 'mybar'` % imprimer en tant que commande
- `>> print('-djpeg', 'mybar')` % imprimer en tant que fonction
- Pour cet exemple, le formulaire de commande génère un résultat involontaire:
- `>> myfile = 'mybar';` % myfile est défini comme une chaîne
- `>> print -djpeg myfile` % en tant que commande, mon fichier est traité comme du texte
- `>> print('-djpeg', myfile)` % en tant que fonction, mon fichier est traité comme une variable
- Les autres utilitaires fréquemment utilisés dans les deux formulaires sont les suivants: `save`, `load`

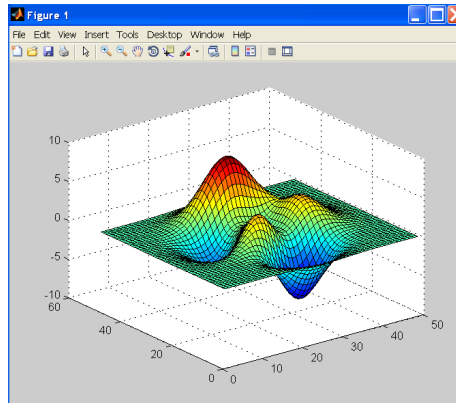
Surface Plot

TP

```
>> Z = peaks; % generate data for plot; peaks returns function values
>> surf(Z)    % surface plot of Z
```

Try these commands also:

```
>> shading flat
>> shading interp
>> shading faceted
>> grid off
>> axis off
>> colorbar
>> colormap('winter')
>> colormap('jet')
```



Exercice d'application

TP

if Conditionnée

Exemple: un programme qui trouve les racine d'une équation 2 degré

```
a=input ('Entrez la valeur de a : '); %lire a
b=input ('Entrez la valeur de b : '); %lire b
c=input ('Entrez la valeur de c : '); %lire c

delta=b^2-4*a*c; %calculer delta
if delta<0
    disp('Pas de solution') %pas de solution
elseif delta==0
    disp('Solution double: ') %solution double
    x=-b/(2*a)
else
    disp('Deux solutions distinctes: ') %deux solution
    x1=(-b+sqrt(delta))/(2*a)
    x2=(-b-sqrt(delta))/(2*a)
end
```

Exercice à rendre

TP

for

Exemple: On rajoute plus 2 a tout le monde :

```
note1 = [ 4 , 12 , 7 , 15 , 12 ]
```

```
note1 = [ 4 , 12 , 7 , 15 , 12 ]
for i = 1:length(note1)
note1(i) = note1(i) + 2;
end
```

Exercice à rendre

TP

for

Exemple: calcul de moyenne sur un vecteur. Ecrire le script suivant:

%on divise la somme des notes par le nombre de notes

```
note2 = [ 4 , 18 , 7 , 15 , 12 ]
```

```
note2 = [ 4 , 18 , 7 , 15 , 12 ]
somme = 0;
for i = 1:length(note2)
%on fait la somme des notes
somme = somme + note2(i);

end
moyenne = somme / length(note2)
```

Exercice à rendre

TP

while

Exemple: On cherche le premier élément négatif d'un vecteur

```
vec = [1,1,1,1,1,1,-1,1,-1]
```

```
vec = [1,1,1,1,1,1,-1,1,-1]
```

```
i = 1;
```

```
while vec(i) >= 0      %tant que vec(i) est superieur a 0
```

```
i = i+1;              %on incremente i pour regarder l'element  
                      suivant de vec
```

```
end
```

```
i                    %indice du premier element negatif
```

```
vec(i)               %premier element negatif
```

Reessayer avec `vec = [1,1,1,1,1,1,1,1,1]`, que ce passe-t-il ?

Exercice à rendre

TP

while

Exemple: On cherche le premier élément négatif d'un vecteur

```
vec = [1,1,1,1,1,1,1,1,1]
```

```
vec = [1,1,1,1,1,1,1,1,1]
```

```
i = 1;
```

```
while (i <= length(vec)) && (vec(i) >= 0)
```

```
i = i+1;
```

```
end
```

```
if i <= length(vec)
```

```
disp( ['i = ', num2str(i)] )
```

```
disp( ['vec(i) = ', num2str( vec(i) ) ] )
```

```
else
```

```
disp('Tous les elements sont positifs')
```

```
end
```

Exercice à rendre

TP

Instructions break

L'instruction break : provoque l'arrêt de la première boucle for, while.

Exemple: L'instruction for dans un programme qui est stoppée au premier i tel que t(i) est nul :

t = -2:10

```
t = -2:10;
for i = 1:length(t)
    if t(i) == 0
        break;
    end
end
disp( ['i = ',num2str(i)] )
disp( ['t(i) = ',num2str( t(i) )] )
```

rappel

TP

fonctions en Matlab

```
function [sortie1,sortie2, ...] =
nom_fonction(entree1,entree2, ...)
...
end
```

Exemple: si on souhaite faire une fonction produit, qui calcule et renvoie en sortie le produit de deux scalaires passés en paramètre, on écrira dans le fichier produit.m :

```
function [res] = produit(a,b)
res = a*b;
end
```

tester cette fonction

e = produit(2,4);

On souhaite afficher la fonction $y = \sin(t)$ sur $0 < t < 2\pi$

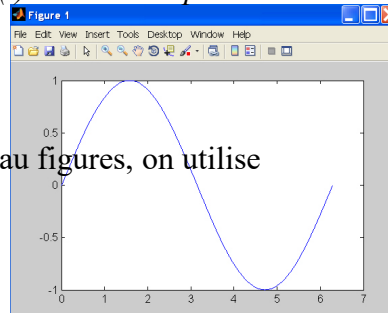
```
>> t = 0:pi/100:2*pi;
>> y = sin(t);
>> plot(t,y)
```

Pour rajouter un titre et des labels d'axes au figures, on utilise

```
>> title('The plot of t vs sin(t)');
>> xlabel('t');
>> ylabel('sin(t)');
```

Style de ligne et couleurs

```
>> plot(t,y,'g.')
>> y2 = sin(t-0.25);
>> y3 = sin(t+0.25);
>> plot(t,y,t,y2,t,y3) % make 2D line plot of 3 curves
>> legend('sin(t)', 'sin(t-0.25)', 'sin(t+0.25)', 1)
```



79

IV- Déclaration de signaux Signaux et Systèmes Discrets

Sinus et exponentielle décroissante

80

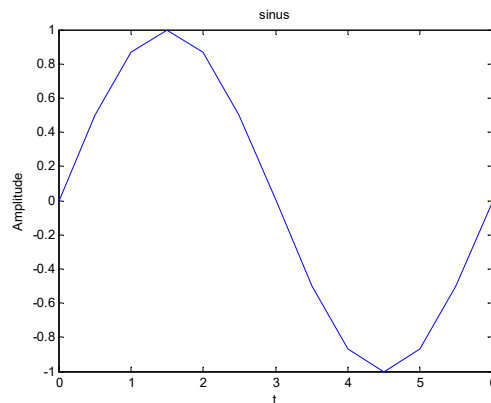
Signaux et Systèmes Discrets

Sinus et exponentielle décroissante

Calcul et tracé d'une fonction sinus, exemple simple

```
%sinus
>> t=0:0.5:6;
>> w = 2*pi/6;
>> y = sin(w*t);
>> plot(t,y,'*');
>> title('sinus');
>> xlabel('t');
>> ylabel('Amplitude');

>> x = -pi : .01 : pi;
>> y = sin(x);
>> plot(x,y)
```



81

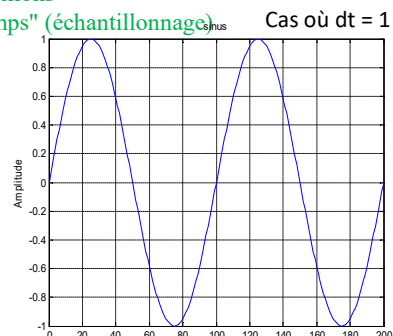
Signaux et Systèmes Discrets

Sinus et exponentielle décroissante

Calcul et tracé d'une fonction sinus, exemple simple

```
%sinus
>> Ts=100;           % Période du sinus
>> duree = 200;      % Durée d'observation signal 2 périodes
>> dt=1;             % Pas ou période d'échantillonnage temporel du signal
>> N=duree/dt;        % Nombre total d'échantillons
>> n=0:N;             % "Vectorisation du temps" (échantillonnage)
>> t=n*dt;            % Définition du temps
>> y=sin(2*pi*t/Ts)    % Génération du sinus
>> plot(t,y,'*');
>> title('sinus');
>> xlabel('t');
>> ylabel('Amplitude');
```

Cas où dt = 50 :

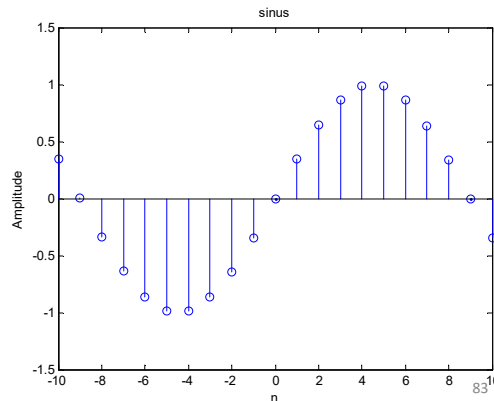


Signaux et Systèmes Discrets

Sinus et exponentielle décroissante

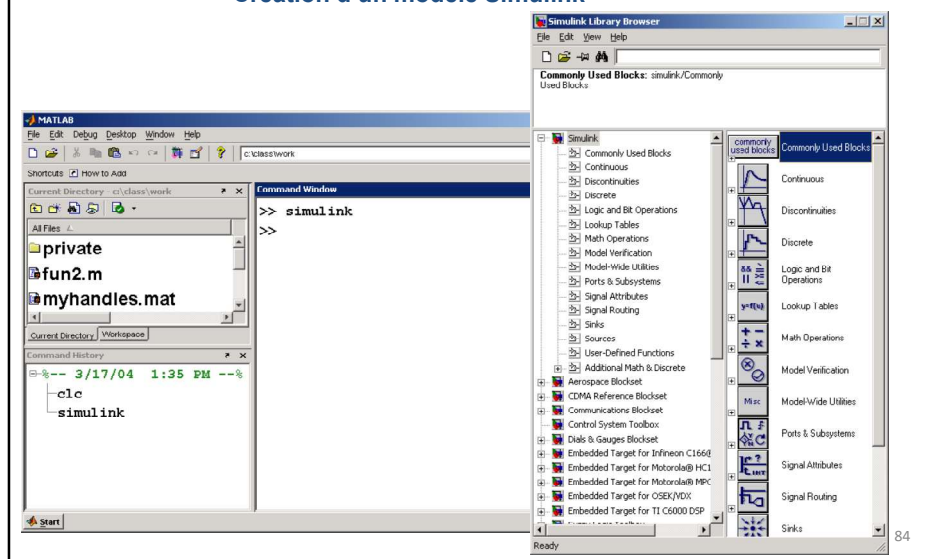
On recommence avec la fonction suivante $\sin(0; 35 \cdot n)$. On écrira le programme suivant :

```
%sinus
t=-10:10;
x=sin(0.35*t);
stem(t,x);
axis([-10 10 -1.5 1.5]);
title('sinus');
xlabel('n');
ylabel('Amplitude');
```



IV- Modélisation par blocs: Simulink

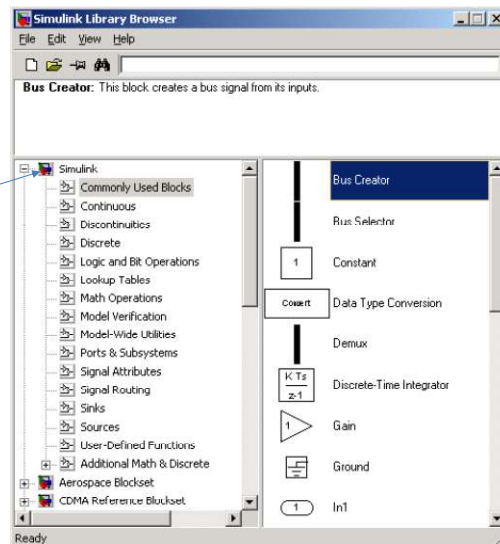
Création d'un modèle Simulink



IV- Modélisation par blocs: Simulink

Explorateur de bibliothèques Simulink

Bibliothèque

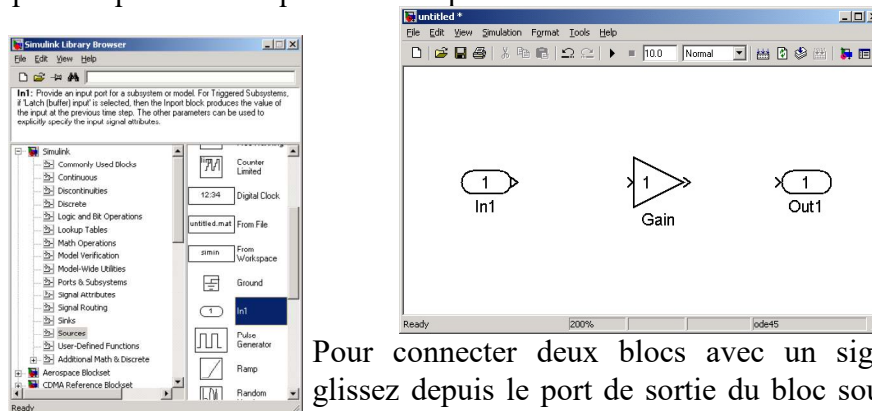


85

IV- Modélisation par blocs: Simulink

Création d'un nouveau schéma : ajout de blocs

Pour ajouter un bloc à un modèle, faites-le glisser dans le schéma-bloc depuis l'explorateur de bibliothèques. Les blocs sont donnés en liste alphabétique dans chaque bibliothèque.



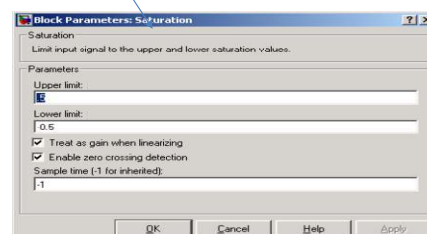
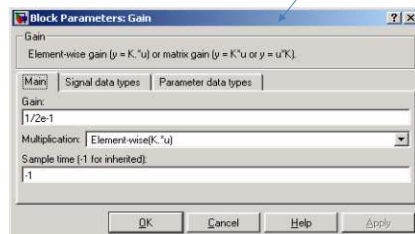
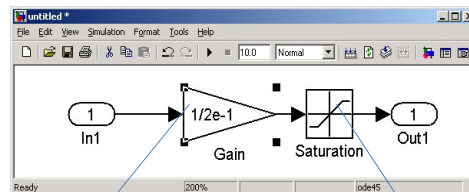
Pour connecter deux blocs avec un signal, glissez depuis le port de sortie du bloc source vers le port d'entrée du bloc cible.

86

IV- Modélisation par blocs: Simulink

Définition des paramètres d'un bloc

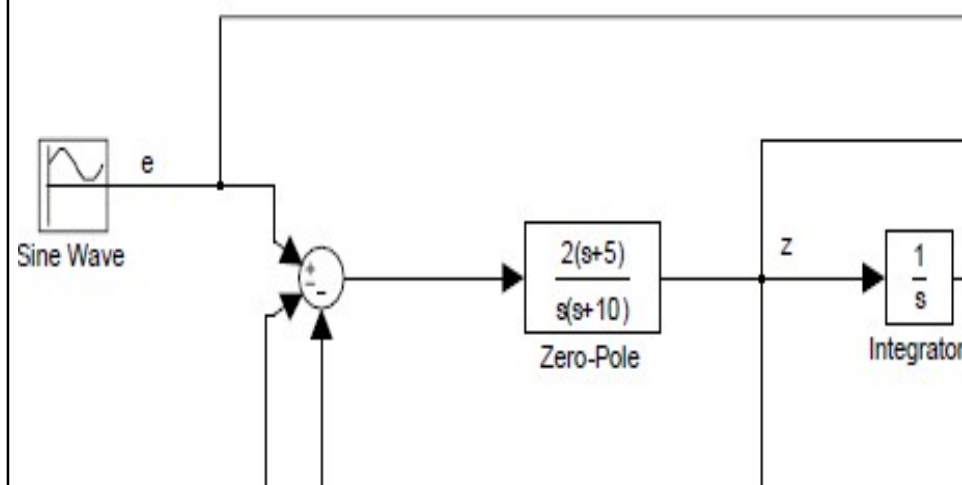
- Pour ouvrir le dialogue des paramètres de bloc, double-cliquez sur le bloc
- Entrez une valeur correcte ou une sélection dans chaque champ.



IV- Modélisation par blocs: Simulink

Exercice d'application

Reproduisez le schéma Simulink de la figure suivante



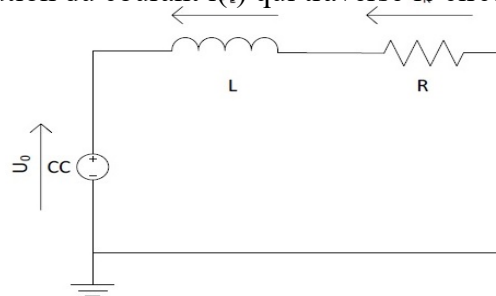
VI- Modélisation par blocs: Simulink

Prenons l'exemple d'un circuit R-L

A $t=0$ s, une source de tension alimente un circuit RL. Le modèle doit permettre d'analyser l'évolution du courant $i(t)$ qui traverse le circuit

$$R = 20 \, \Omega$$

$$L = 0.01 \, \text{mH}$$



Les tensions dans le circuit sont liées par la relation temporelle suivant:

$$U(t) = UR(t) + UL(t) = Ri(t) + L \frac{di(t)}{dt}$$

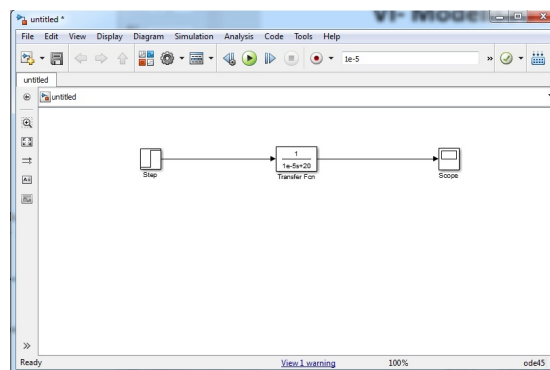
En appliquant la TL à cette equation :

$$U(p) = (R + Lp)I(p) \quad \Rightarrow \quad I(p) = \frac{1}{(R + Lp)} U(p)$$

89

VI- Modélisation par blocs: Simulink

Choix des composants:



modèle du circuit R-L réalisé avec
Simulink

90

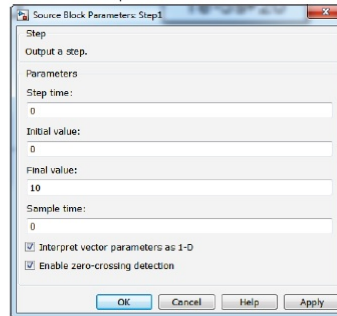
VI- Modélisation par blocs: Simulink

Paramétrage des composants:

STEP



Simulink/Sources



Step time: instant à partir duquel l'échelon passe à la valeur initiale, ici à $t=0$ s.

Initial value: valeur initiale de l'échelon; ici 0

Final value: valeur finale de l'échelon qui correspond ici à la tension de 10 V que l'on souhaite imposer.

Sample time: toujours laisser cette valeur à 0 pour l'étude des systèmes à temps continu.

91

VI- Modélisation par blocs: Simulink

TRANSFERT FUNCTION



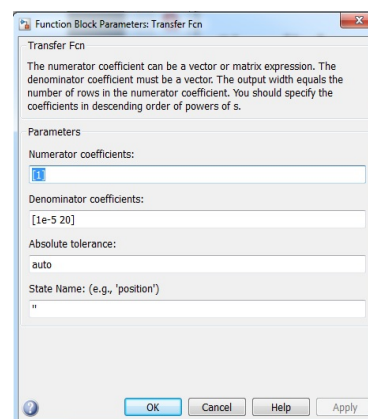
Transfer Fcn

La fonction de transfert à saisir est

$$H(p) = \frac{1}{(R + Lp)} = \frac{1}{1e^{-5}p + 20}$$

Avec $R=20 \Omega$

$L=0.01$ mH



Le paramétrage des fonctions de transfert se fait par la saisie des coefficients du numérateur et du dénominateur en commençant par le coefficient du monôme de plus haut degré.

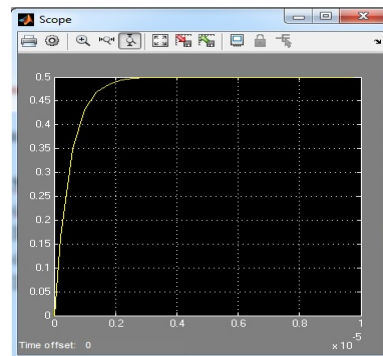
92

VI- Modélisation par blocs: Simulink

TP

Lancement de la simulation et analyse des résultats

- **Spécifier** un temps de simulation de **1e-5 seconde** et **lancer** la simulation en cliquant sur la fleche verte.
- Faire un double cli sur le scope qui mesure le courant pour obtenir la variation du courant en fonction du temps.
- La mise à l'échelle



La courbe donnant l'évolution du courant dans le circuit.

93

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Polynômes

Un polynôme est introduit sous la forme d'un vecteur-ligne dont les composantes représentent les coefficients des puissances décroissantes.

```
>> P1=[1, 2, 4, 7];      % définition du polynôme  $s^3 + 2s^2 + 4s^1 + 7$ 
>> P2=[ 1 0 4];         % définition du polynôme  $s^2 + 4$ 
>> Rac= roots(P1);       % calcul les racines de P1
>> P1=poly(Rac);         % à partir des racines on obtient le polynôme correspondant
>> abs(roots(P1))        % calcul du module (abs) des racine (roots) de P1
>> Pod=conv(P1,P2);      % multiplication:  $P1 \cdot P2 = (s^3 + 2s^2 + 4s^1 + 7)(s^2 + 4)$ .
```

94

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Polynômes

On définit le polynôme:

$$p(s) = 6s^6 + 5s^5 + 3s^3 + 2s^2 + s$$

avec l'instruction suivante :

```
>> P=[6 5 0 3 2 1 0]
```

On obtient les racines de ce polynôme avec la fonction **roots**

```
>> roots(P)
```

On désire stocker la fonction de transfert

$$F(s) = \frac{s^2 + s}{s^3 + 3s^2 + s + 1}$$

Il y a deux façons de définir une telle fonction de transfert en utilisant la fonction tf.

95

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Polynômes

```
>> F=tf([1 1 0],[1 3 1 1])
```

```
>> s=tf('s')
```

```
>> F=(s^2+s)/(s^3+3*s^2+s+1)
```

Obtenir les pôles, zéros et gain statique de la fonction de transfert et factoriser la fonction de transfert.

```
>> pole(F)
```

```
>> zero(F)
```

```
>> dcgain(F)
```

```
>> zpk(F)
```

96

IV- Déclaration de signaux

Signaux et Systèmes Discrets

fonctions de transfert

La fonction de transfert $G_1(p) = \frac{p+2}{3p^2+6p+1}$ sera mise en mémoire grâce à l'instruction :

```
>> sys1 = tf([1 2],[3 6 1]);
>> sys1
sys1 =
      s + 2
-----
3 s^2 + 6 s + 1
Continuous-time transfer function.
>>
```

Entrer les fonctions de transfert suivantes

$$G_2(p) = \frac{0.5}{0.05p + 1}$$

$$G_3(p) = \frac{1}{p + 1}$$

$$G_4(p) = \frac{2.5}{p^2 + 0.5p + 1}$$

97

IV- Déclaration de signaux

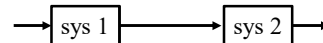
Signaux et Systèmes Discrets

Opération sur les fonctions de transfert

Il est possible d'assembler 2 fonctions de transfert entre elles :

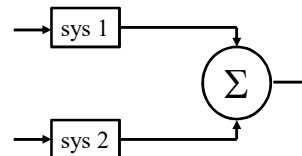
En série :

```
>> sys_serie = sys1 * sys2;
```



En parallèle :

```
>> sys_para=sys1 + sys2;
```



98

IV- Déclaration de signaux

Signaux et Systèmes Discrets

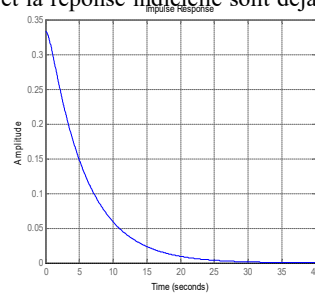
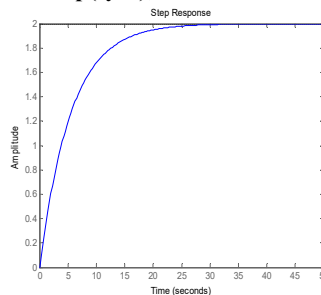
Opération sur les fonctions de transfert

Représentation temporelle

A partir d'une fonction de transfert, Matlab peut calculer la réponse temporelle d'une fonction de transfert à une entrée. La réponse impulsionnelle et la réponse indicielle sont déjà programmées dans des fonctions.

- La réponse impulsionnelle ($e(t) = \delta(t)$) à l'aide de la fonction
>>impulse(sys1);

- La réponse indicielle ($e(t) = u(t)$) à l'aide de la fonction
>>step(sys1);



En cliquant sur le tracé avec le bouton droit, il est possible de modifier les propriétés du graphe et surtout d'obtenir des renseignements sur les temps de réponse ou dépassement du tracé.

Tracer la réponse impulsionnelle et indicielle de sys2 et déterminer de manière graphique la valeur de la sortie en régime permanent et la valeur de la constante de temps.

Tracer ces deux réponses pour sys4.

99

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Etude de cas

Système du second ordre

Nous allons d'abord étudier un système du second ordre :

$$H(p) = \frac{10}{s^2 + 2.s + 10}$$

Ecriture de la fonction de transfert :

```
>> num=10
>> den=[1 2 10]
>> printsys(num,den)
```

Etude de la réponse à un échelon :

```
>> step(num,den)
```

100

IV- Déclaration de signaux

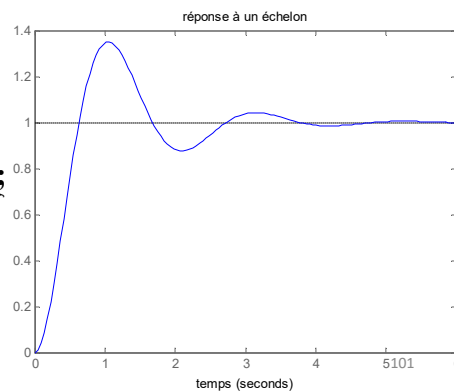
Signaux et Systèmes Discrets

pour voir la réponse à un échelon pendant 10s

```
>> t=0:0.1:10 ;
>> y=step(num,den,t); le point virgule ";" évite d'afficher le résultat
>> plot(t,y)
```

Commentaires sur la courbe :

```
>> title('réponse à un échelon');
>> xlabel('temps');
>> ylabel('y');
```



IV- Déclaration de signaux

Signaux et Systèmes Discrets

Pour lire des valeurs sur la courbe :

```
>> [x,y]=ginput(3) et cliquer avec la souris sur 3 points à mesurer
```

Pour tracer un quadrillage ou une ligne :

```
>> grid
>> line([0 10],[1.05 1.05])
>> line([0 10],[1.05 1.05] , 'color','g')
```

ces instructions permettent de tracer deux lignes horizontales et de faire apparaître le "tuyau des réponses à 5%"

```
>> line([0 10],[1.05 1.05])
>> line([0 10],[0.95 0.95])
```

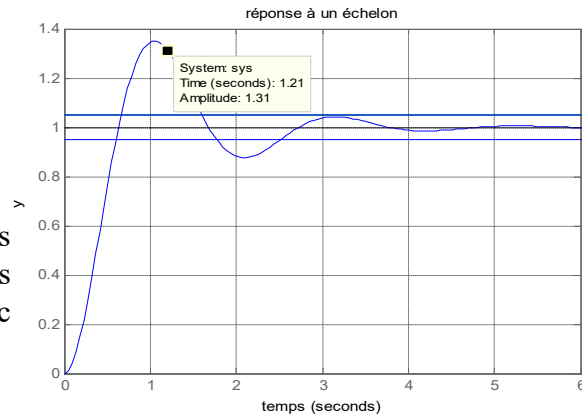
la commande line([x1,x2],[y1,y2]) trace une ligne du point (x1,y1) au point (x2,y2).

102

IV- Déclaration de signaux

Signaux et Systèmes Discrets

On peut mesurer les coordonnées de points intéressants avec l'instruction *ginput* :



le temps de réponse à 5%
le dépassement D%
le temps de montée au premier pic t_{pic}
la pseudo période.

103

IV- Déclaration de signaux

Signaux et Systèmes Discrets

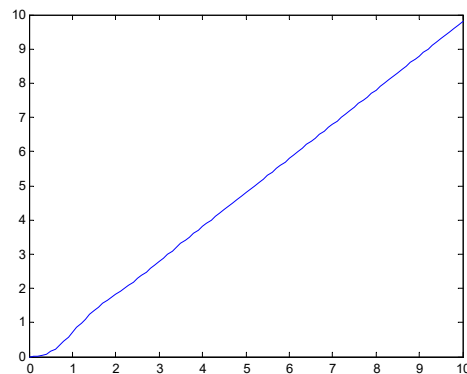
Réponse à un signal quelconque

D'abord définir le temps :

>> $t=0:0.1:10$; la variable t prend des valeurs de 0 à 10 par pas de 0,1

une rampe :

>> ramp=t;
>> y=lsim(num,den,ramp,t);
>> plot(t,y)



104

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Réponse à un signal quelconque

Une sinusoïde :

```
>> sinus=sin(t);           (ne pas employer sin comme nom car
                             c'est une fonction de Matlab et cela
                             entraînerait une "plante" du programme)
>> z=lsim(num,den,sinus,t);
>> plot(t,z)
```

Afin de visualiser la sinusoïde originale et le signal de sortie, taper :

```
>> hold on
>> plot(t,sinus,'r')      'r' permet d'affecter la couleur rouge à ce
                             tracé pour le différencier du précédent
```

105

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Réponse à un signal quelconque

Pour construire un système en boucle fermée :

```
>> [numf,denf]=cloop(num,den,-1);
>> step(numf,denf)         permet de visualiser la réponse en boucle
                             fermée unitaire à un échelon
```

L'influence d'un correcteur. Par exemple, si on considère un simple gain de 6, il faut redéfinir le numérateur par l'instruction :

```
>> num=60 (10×6)
```

Redéfinir la fonction de transfert en boucle fermée :

```
>> [numf,denf]=cloop(num,den,-1);
```

et faire dessiner la réponse à un échelon

```
step(numf,denf)
```

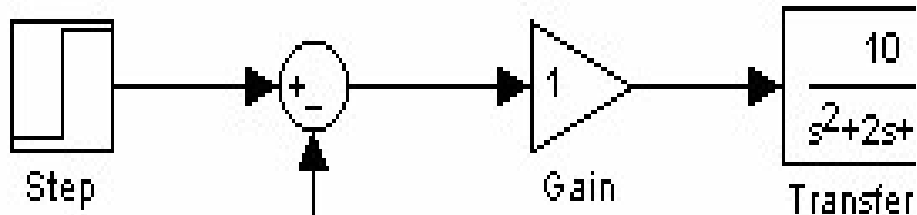
106

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Analyse de H à l'aide de simulink

➤ Construire H, Raccorder les éléments comme indiqué ci-dessous.



Ajuster les coefficients du numérateur et du dénominateur "transfer Fcn".

L'échelon peut être réglé lui aussi en double cliquant dessus. Par défaut, il possède une amplitude de 1 et devient égal à 1 à $t = 0,5$ s.

107

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Définition

Exemple

a- Signaux analogiques: sont des fonctions définies pour toutes valeurs de la variable. Pour étudier ce type de signaux, on utilise des modèles mathématiques qui se prêtent bien à l'étude.

c- Principaux signaux: Pour tester la plus part des systèmes électroniques de base, nous faisons appel aux signaux élémentaires suivantes:

Fonction porte ou fenêtre
Fonction échelon
Fonction impulsion de Dirac
Fonctions périodiques

108

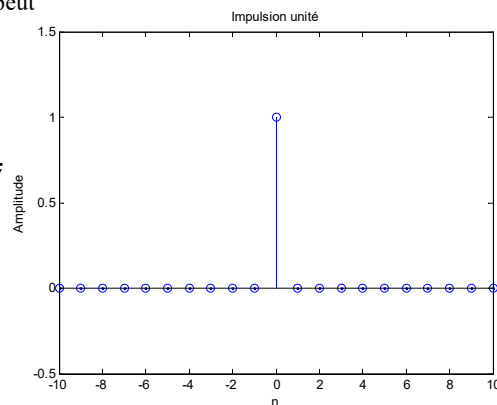
IV- Déclaration de signaux

Signaux et Systèmes Discrets

Fonction impulsion unité

Afin de générer l'impulsion unité on peut écrire le programme suivant :

```
%impulsion unité
t=-10:10;
x=[zeros(1,10),1,zeros(1,10)];
stem(t,x);
axis([-10 10 -0.5 1.5]);
title('Impulsion unité');
xlabel('n');
ylabel('Amplitude');
```



Ecrivez et testez le programme précédent. Expliquez à quoi servent les différentes fonctions.

109

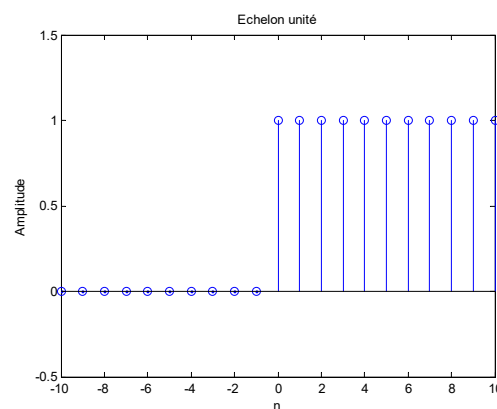
IV- Déclaration de signaux

Signaux et Systèmes Discrets

Fonction échelon unité

Pour le cas de l'échelon unité, on se contentera d'un nombre fini d'échantillon. On peut écrire le programme suivant :

```
%échelon unité
t=-10:10;
x=[zeros(1,10),ones(1,11)];
stem(t,x);
axis([-10 10 -0.5 1.5]);
title('Echelon unité');
xlabel('n');
ylabel('Amplitude');
```



Ecrivez et testez le programme précédent.

110

IV- Déclaration de signaux

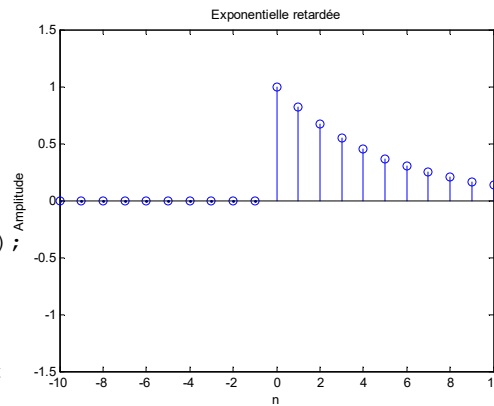
Signaux et Systèmes Discrets

Sinus et exponentielle décroissante

Ensuite avec une exponentielle décroissante $e^{0.2 \cdot n} * u[n]$ avec u l'échelon unité. On a le programme suivant :

```
%exponentielle
t=-10:10;
u=[zeros(1,10),ones(1,11)];
x=exp(-0.2*t).*u;
stem(t,x);
axis([-10 10 -1.5 1.5]);
title('Exponentielle retardée');
xlabel('n');
ylabel('Amplitude');
```

`stem(t,x);` plots the data sequence, Y, at values specified by X



On utilise ici l'opérateur de multiplication termes à termes `.*` qui permet d'effectuer la multiplication terme à terme de deux matrices $((A .* B)_{ij} = A_{ij} * B_{ij})$.

111

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Transformée de Laplace

La commande `laplace` permet d'obtenir très facilement la transformée de Laplace d'une fonction.

```
%Transformée de Laplace d'une expression
syms t;
Saisir le script      syms w;
suivant               syms a;
                      laplace (sin(w*t))
                      laplace (cos(w*t))
                      laplace (exp(-a*t)*cos(w*t))
                      laplace (exp(-a*t)*sin(w*t))
```

script permettant de calculer la transformée de Laplace de fonction

```
syms t;               %Crée une ou plusieurs variables symboliques.
                      Ne permet de créer des constantes symboliques.
```

112

IV- Déclaration de signaux

Signaux et Systèmes Discrets

L'exécution du script permet d'obtenir le résultat suivant:

```
>> transfo

ans =
w/(s^2 + w^2)          calcul de la TL de sin(wt)

ans =
s/(s^2 + w^2)          calcul de la TL de cos(wt)

ans =
(a + s)/((a + s)^2 + w^2)  calcul de la TL de  $e^{-at} \cos(\omega t)$ 

ans =
w/((a + s)^2 + w^2)

>>
```

113

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Transformée inverse de Laplace

La commande *ilaplace* permet d'obtenir très facilement la transformée de Laplace d'une fonction.

Saisir le script suivant

```
%%Transformée de Laplace d'une expression
syms t;
syms w;
syms a;

%%Transformée inverse de Laplace d'une expression
syms s;
ilaplace (s/(s^2 + w^2))
ilaplace (3/(2+s) - 5/(3+s)^2)
```

script permettant d'obtenir la transformée inverse de Laplace d'une fonction

114

IV- Déclaration de signaux

Signaux et Systèmes Discrets

L'exécution du script permet d'obtenir le résultat suivant:

```
>> transfo_inv

ans =

cos(t*w)                                calcul de la transformée inverse de Laplace de  $\frac{s}{s^2+\omega^2}$ 

ans =

3*exp(-2*t) - 5*t*exp(-3*t)             calcul de la transformée inverse de Laplace de  $\frac{3}{2+s} - \frac{5}{3s^2}$ 

>>
```

115

IV- Déclaration de signaux

Signaux et Systèmes Discrets

Transformée de Laplace & inverse L

```
syms s t;                                %Crée une ou plusieurs variables
                                          symboliques. Ne permet de créer
                                          des constantes symboliques.

laplace(sin(4*t))                        % to find the Laplace transform of a
                                          time function.

syms s t;
Fs=(3*s+2)/(s^2+3*s+2);
ft=ilaplace(Fs);                         % to obtain the time domain function
                                          directly from F(s).

pretty(ft)
```

script permettant de calculer la transformée de Laplace et inverse d'une fonction

IV- Déclaration de signaux

Signaux et Systèmes Discrets

L'exécution du script permet d'obtenir le résultat suivant:

```
>> lapla1

ans =

4/(s^2 + 16)

exp(-2 t) 4 - exp(-t)

>>
```

117

Application : Simulink

TP

Jeux de lumière :

```
a = arduino('COM4');

for k=1:10
    disp('turn on LED');
    a.pinMode(13,'output')
    a.digitalWrite(13,1);
    pause(1);
    disp('turn of LED');
    a.digitalWrite(13,0);
    pause(1);
    disp('turn on LED');
    a.pinMode(10,'output')
    a.digitalWrite(10,1);
    pause(1);
    disp('turn of LED');
    a.digitalWrite(10,0);
    pause(1);
    disp('turn on LED');
    a.pinMode(7,'output')
    a.digitalWrite(7,1);
    pause(1);
    disp('turn of LED');
    a.digitalWrite(7,0);
    pause(1);
end
disp('close Arduino board');
clear a;
```

118

Application : Simulink

TP

LED avec BP :

```
function [] = led_pushbutton()
    pushbutton = 'D10';
    led = 'D7';
    a = arduino();
    finishup = onCleanup(@() exitprogram(a));
    a.pinMode(10,'Input');
    disp('press Ctr-C to exit');
    while 1
        val=a.digitalRead(10);
        a.digitalWrite(7,val);
        disp(val);
        pause(0.5);
    end
end
function exitprogram(b)
    clear b;
    disp('program has exit');
end
```

%LED is connected to Arduino digital pin 7
%Pushbutton is connected to Arduino digital pin 10

Press pushbutton. Then, you should see lighting LED. Press CTRL+C to exit program.

Program output:

LED is lighting while a pushbutton is pressed.

119